Rapport de Projet

Analyse Comportementale Clientèle Retail

Atelier Pratique — E-commerce de Cadeaux

Étudiant(e) : Rim Chaieb

Classe : GI2S2

Encadrante : Mme Fadoua Drira

Année : 2025–2026

22 avril 2026

Résumé

Ce rapport présente la réalisation complète d'un atelier de Machine Learning appliqué à l'analyse comportementale d'une clientèle e-commerce spécialisée dans la vente de cadeaux.

L'objectif principal est de construire une chaîne de traitement complète allant de l'exploration des données brutes jusqu'au déploiement d'une application web interactive. Le jeu de données comprend **52 features** décrivant le comportement transactionnel, démographique et relationnel de chaque client.

Les travaux réalisés couvrent cinq étapes majeures :

1. **Exploration** : analyse de la structure, des distributions et de la qualité des données
2. **Préparation** : nettoyage, imputation, encodage et normalisation
3. **Modélisation** : classification (prédiction du churn) et régression
4. **Évaluation** : métriques de performance et interprétation des résultats
5. **Déploiement** : interface web Flask pour la prédiction en temps réel

Résultats clés : Le modèle Random Forest optimisé (GridSearchCV) obtient une accuracy de **92.69%** et un F1-score de **0.9046** sur le jeu de test. Le modèle de régression atteint un R^2 de **90.13%** pour la prédiction des revenus clients.

Mots-clés : Machine Learning, Churn Prediction, Random Forest, Analyse RFM, K-Means, ACP, Flask, Scikit-learn, E-commerce.

Table des matières

Résumé **1**

Table des matières **2**

Table des figures **4**

Liste des tableaux **5**

1 Introduction **6**
1.1 Contexte du projet 6
1.2 Objectifs 6
1.3 Structure du rapport 6
1.4 Environnement technique 7

2 Exploration des Données **8**
2.1 Présentation du jeu de données 8
2.2 Analyse descriptive 9
2.2.1 Statistiques générales et distributions 9
2.2.2 Distribution de la variable cible (Churn) 10
2.2.3 Distributions des features numériques 11
2.2.4 Analyse des variables catégorielles 12
2.3 Analyse de la qualité des données 13
2.3.1 Valeurs manquantes 13
2.4 Analyse des corrélations 14

3 Préparation des Données **16**
3.1 Pipeline de prétraitement 16
3.2 Nettoyage des données 17
3.2.1 Traitement des valeurs aberrantes 17
3.2.2 Parsing des dates 17
3.2.3 Suppression des features inutiles et colonnes fuite 17
3.3 Imputation des valeurs manquantes 18
3.4 Encodage des variables catégorielles 19
3.5 Feature Engineering 19
3.6 Séparation Train/Test et Normalisation 19
3.7 Gestion du déséquilibre des classes (SMOTE) 20
3.7.1 Motivation 20

3.7.2 Résultats après SMOTE 20

4 Segmentation — K-Means & ACP **21**

4.1 Objectif de la segmentation 21

4.2 Réduction de Dimension — ACP 21

4.2.1 Motivation et Variance Expliquée 21

4.3 Segmentation des clients — K-Means 21

4.3.1 Méthode du Coude (Elbow Method) 21

4.3.2 Répartition des 4 segments 22

4.3.3 Visualisation 2D des clusters 22

5 Modélisation **24**

5.1 Tâche 1 — Classification : Prédiction du Churn 24

5.1.1 Modèles testés 24

5.1.2 Optimisation des hyperparamètres (GridSearchCV) 24

5.1.3 Évaluation du modèle de classification 25

5.1.4 Importance des features 26

5.2 Tâche 2 — Régression : Prédiction des Revenus 27

5.2.1 Objectif et features utilisées 27

5.2.2 Évaluation du modèle de Régression 27

6 Déploiement — Application Flask **28**

6.1 Architecture de l'application 28

6.2 Structure du projet 28

6.3 Fonctionnement de l'application 29

6.4 Interface utilisateur 30

7 Conclusion **31**

7.1 Bilan des travaux réalisés 31

7.2 Résultats clés 31

7.3 Limites identifiées 32

7.4 Perspectives 32

Table des figures

2.1 Statistiques descriptives des features numériques principales 9
2.2 Distribution de la variable cible Churn 10
2.3 Distribution des principales features numériques 11
2.4 Distribution des principales features catégorielles 12
2.5 Analyse des valeurs manquantes par feature (avant nettoyage) 13
2.6 Analyse des valeurs manquantes par feature (après nettoyage) 13
2.7 Matrice de corrélation des features numériques (heatmap complète) . . 14
2.8 Zoom sur les corrélations les plus fortes 15

3.1 Pipeline séquentiel de prétraitement des données 16
3.2 Distribution asymétrique des variables numériques avant transformation logarithmique 18
3.3 Distribution de la variable cible avant et après SMOTE 20

4.1 Méthode du Coude — détermination du nombre optimal de clusters . . 22
4.2 Profils comportementaux des 4 segments clients 22
4.3 Projection des segments clients sur les deux premiers axes factoriels (ACP) 23

5.1 Matrice de confusion — Random Forest optimisé (GridSearchCV) . . . 25
5.2 Top 15 features — importance pour la prédiction du churn (Random Forest GridSearch) 26

6.1 Architecture de l'application Flask 28
6.2 Interface web Flask — Churn Predictor 30

Liste des tableaux

1.1 Stack technologique du projet 7

2.1 Répartition des features par type 8
2.2 Résumé des problèmes de qualité identifiés 14

3.1 Évolution du nombre de features à travers le pipeline 17
3.2 Stratégies d'imputation par feature 18
3.3 Encodage des variables catégorielles 19

4.1 Répartition des clients par cluster K-Means 22

5.1 Modèles de classification comparés 24
5.2 Métriques de classification — comparaison des modèles 25
5.3 Métriques de régression — comparaison des modèles 27

6.1 Features de l'interface Flask 29

7.1 Récapitulatif des étapes réalisées 31

Chapitre 1

Introduction

1.1 Contexte du projet

Dans un marché e-commerce de plus en plus concurrentiel, la rétention clientèle constitue un enjeu stratégique majeur. Le coût d'acquisition d'un nouveau client est estimé entre 5 et 25 fois supérieur à celui de la fidélisation d'un client existant. Il devient donc essentiel pour les entreprises de détecter précocement les signaux de départ (*churn*) afin d'engager des actions correctives personnalisées.

Ce projet s'inscrit dans ce contexte en appliquant des techniques de Machine Learning à une base de données réelle d'un e-commerce de cadeaux. Nous disposons de **52 features** couvrant les dimensions transactionnelles, comportementales et démographiques de la clientèle.

1.2 Objectifs

Objectifs du projet

1. Maîtriser la chaîne complète de traitement en data science
2. Appliquer les techniques de preprocessing sur des données imparfaites
3. Construire et évaluer des modèles de classification et de régression
4. Déployer une application web interactive avec Flask

1.3 Structure du rapport

Ce rapport est organisé en six chapitres suivant la progression naturelle d'un projet de data science : exploration, préparation, modélisation, évaluation et déploiement. Chaque étape est documentée avec les choix techniques effectués et leur justification.

1.4 Environnement technique

Table 1.1 – Stack technologique du projet

Outil	**Version**	**Usage**
Python	3.10+	Langage principal
Pandas	2.x	Manipulation des données
Scikit-learn	1.x	Modèles ML et preprocessing
Matplotlib	3.x	Visualisations
Seaborn	0.x	Visualisations statistiques
Imbalanced-learn	–	SMOTE (rééquilibrage des classes)
Flask	3.x	Application web
Joblib	–	Sérialisation des modèles
VS Code	–	Environnement de développement

Chapitre 2

Exploration des Données

2.1 Présentation du jeu de données

Le dataset `retail_customers_COMPLETE_CATEGORICAL.csv` contient des données transactionnelles issues d'un e-commerce de cadeaux. Il regroupe 52 features réparties en trois catégories principales :

Table 2.1 – Répartition des features par type

Type	**Nombre**	**Exemples**
Numériques continues	20	MonetaryTotal, Age, WeekendRatio
Numériques entières	14	Frequency, Recency, TotalTransactions
Catégorielles	18	RFMSegment, LoyaltyLevel, Region
Total	**52**	–

Après chargement et preprocessing complet, le pipeline produit **58 features** finales (après encodage one-hot et feature engineering), réparties en 3 497 échantillons d'entraînement et 875 de test.

2.2 Analyse descriptive

2.2.1 Statistiques générales et distributions

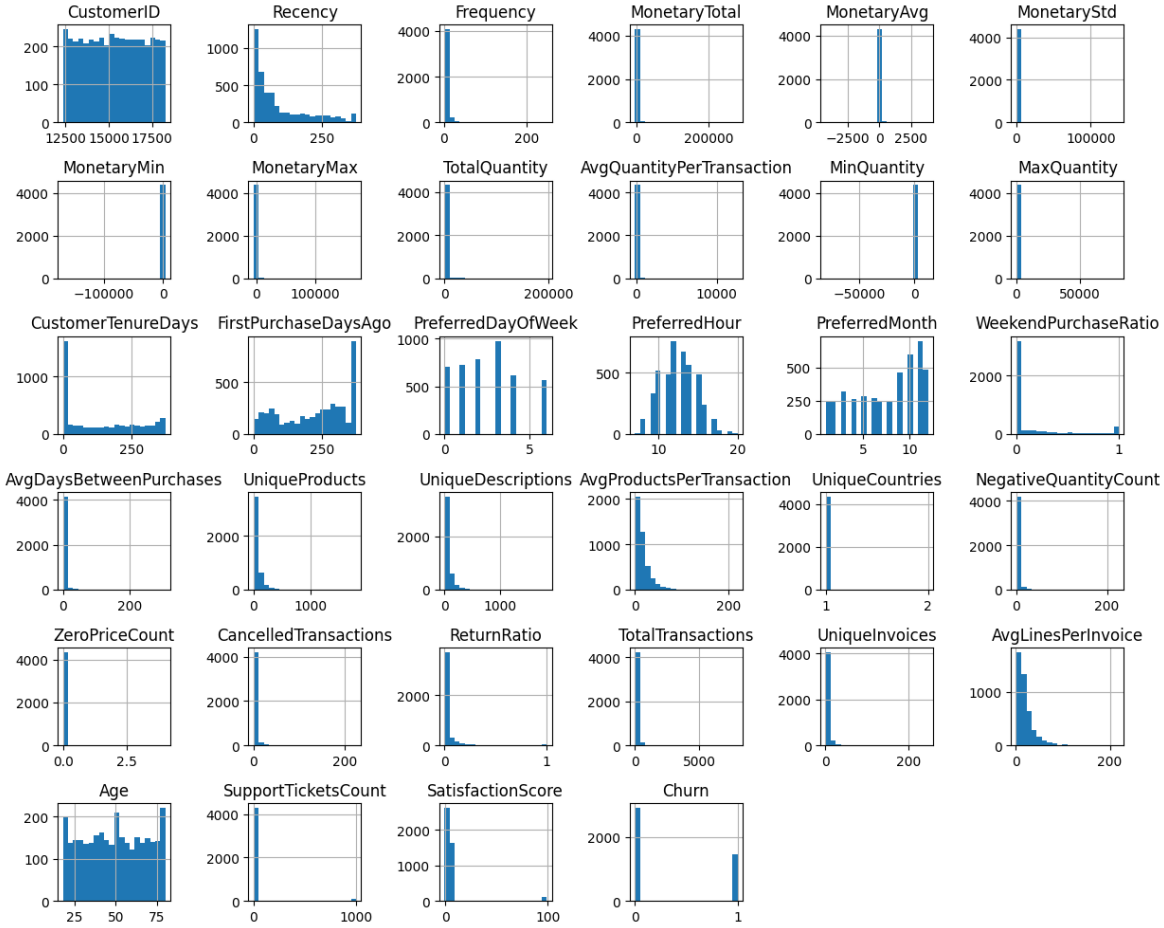


FIGURE 2.1 – Statistiques descriptives des features numériques principales

Les statistiques descriptives révèlent plusieurs observations importantes. La variable `MonetaryTotal` présente une forte asymétrie positive (skewness élevé) indiquant la présence de clients à très forte valeur. La variable `Recency` montre une distribution étalée, témoignant d'une hétérogénéité dans l'activité récente des clients.

2.2.2 Distribution de la variable cible (Churn)

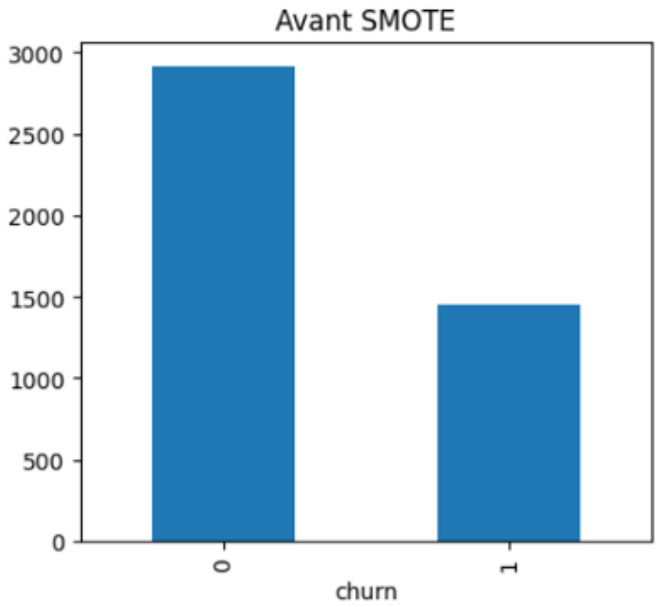


FIGURE 2.2 – Distribution de la variable cible Churn

La variable cible `Churn` présente un déséquilibre de classes notable : **66.7%** de clients fidèles (Churn=0, soit 584 dans le jeu de test) contre **33.3%** de clients partis (Churn=1, soit 291 dans le jeu de test). Ce déséquilibre justifie l'application de SMOTE lors de l'entraînement ainsi que l'utilisation du paramètre `class_weight='balanced'` dans nos modèles.

2.2.3 Distributions des features numériques

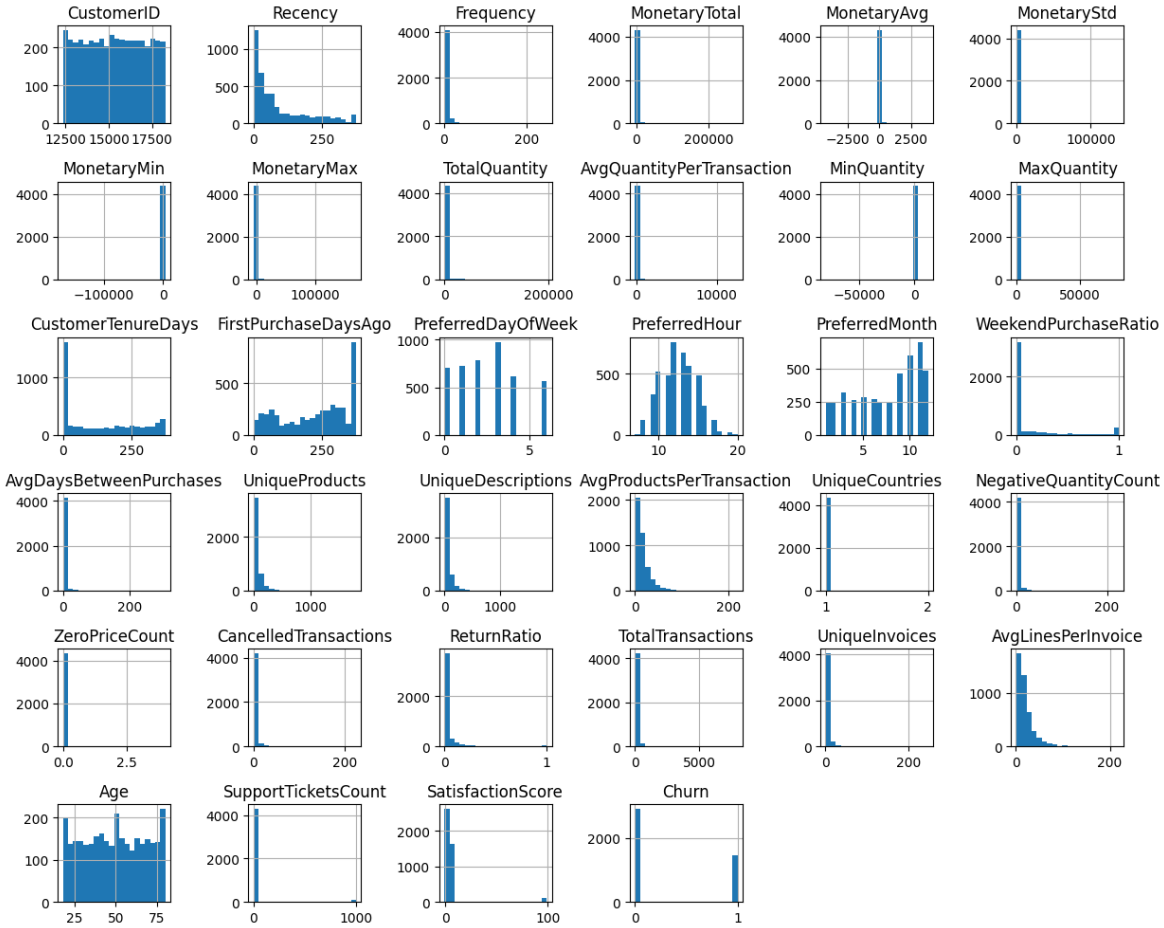


FIGURE 2.3 – Distribution des principales features numériques

L'analyse des histogrammes met en évidence :

— **Recency** : distribution asymétrique à droite — la majorité des clients ont acheté récemment

— **Frequency** : distribution concentrée sur les faibles valeurs avec une longue queue droite

— **MonetaryTotal** : forte asymétrie positive, quelques clients très dépensiers

— **Age** : distribution approximativement normale avec des valeurs manquantes (≈30%)

— **SatisfactionScore** : pic sur les valeurs aberrantes (99, -1) à corriger lors du preprocessing

Ces observations soulèvent trois besoins de preprocessing majeurs :

— **Gestion de l'asymétrie et des échelles :** La majorité des variables présentent une forte asymétrie positive et des ordres de grandeur disparates. Afin de ne

pas biaiser les algorithmes sensibles aux distances (KNN, SVM), une **transformation logarithmique** ($\log 1p$) sera appliquée, suivie d'une **standardisation (StandardScaler)**.

— **Traitement des anomalies et valeurs extrêmes :** La présence d'outliers (dépenses massives) et de valeurs aberrantes (scores de satisfaction à 99 ou -1) nécessite un nettoyage ciblé par **clipping ou suppression** pour stabiliser l'apprentissage.

— **Intégrité et cohérence des données :** Le taux important de valeurs manquantes (30% sur l'`Age`) et la présence de valeurs négatives (potentiels retours produits) feront l'objet d'une **imputation spécifique** pour distinguer les erreurs des transactions valides.

Note technique : Ces étapes de preprocessing sont essentielles pour transformer des données brutes hétérogènes en un signal exploitable, garantissant ainsi la convergence et la robustesse des modèles de Machine Learning.

2.2.4 Analyse des variables catégorielles

```
Colonnes catégorielles : Index(['RFMSegment', 'AgeCategory', 'SpendingCategory', 'CustomerType',
       'FavoriteSeason', 'PreferredTimeOfDay', 'Region', 'LoyaltyLevel',
       'ChurnRiskCategory', 'WeekendPreference', 'BasketSizeCategory',
       'ProductDiversity', 'Gender', 'AccountStatus', 'Country'],
      dtype='str')
```

FIGURE 2.4 – Distribution des principales features catégorielles

2.3 Analyse de la qualité des données

2.3.1 Valeurs manquantes

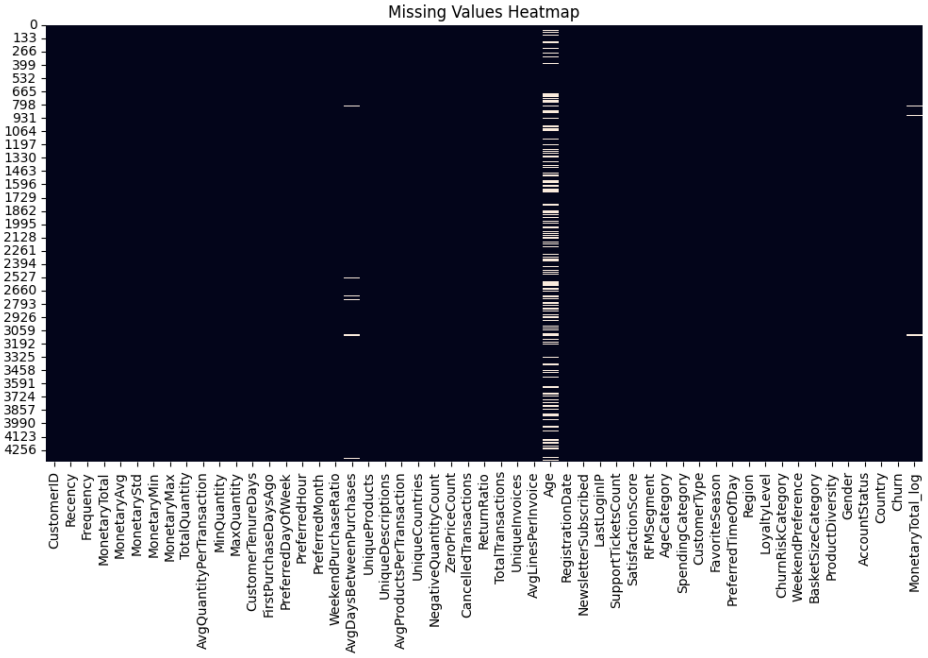


FIGURE 2.5 – Analyse des valeurs manquantes par feature (avant nettoyage)

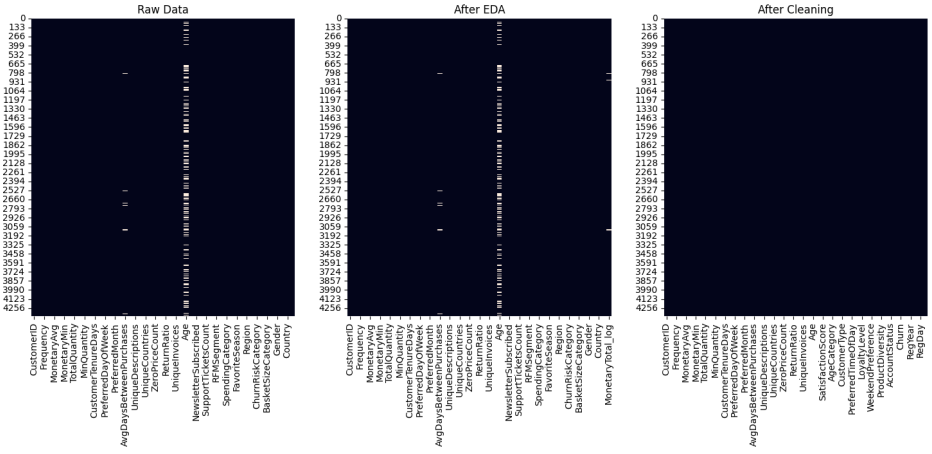


FIGURE 2.6 – Analyse des valeurs manquantes par feature (après nettoyage)

Note : Les valeurs aberrantes sont d'abord remplacées par `NaN`, puis imputées par la médiane. Cette approche en deux temps garantit une imputation robuste non biaisée par les valeurs extrêmes.

TABLE 2.2 – Résumé des problèmes de qualité identifiés

Feature	**Problème**	**Taux**
Age	Valeurs manquantes (NaN)	≈30%
SatisfactionScore	Valeurs aberrantes (-1, 99)	≈5–8%
SupportTicketsCount	Valeurs aberrantes (-1, 999)	≈5–8%
RegistrationDate	Formats inconsistants (UK/ISO/US)	100%
NewsletterSubscribed	Constante (toujours "Yes")	100%
LastLoginIP	Données brutes à parser	100%
AccountStatus / Churn	Déséquilibre de classes	Variable

2.4 Analyse des corrélations

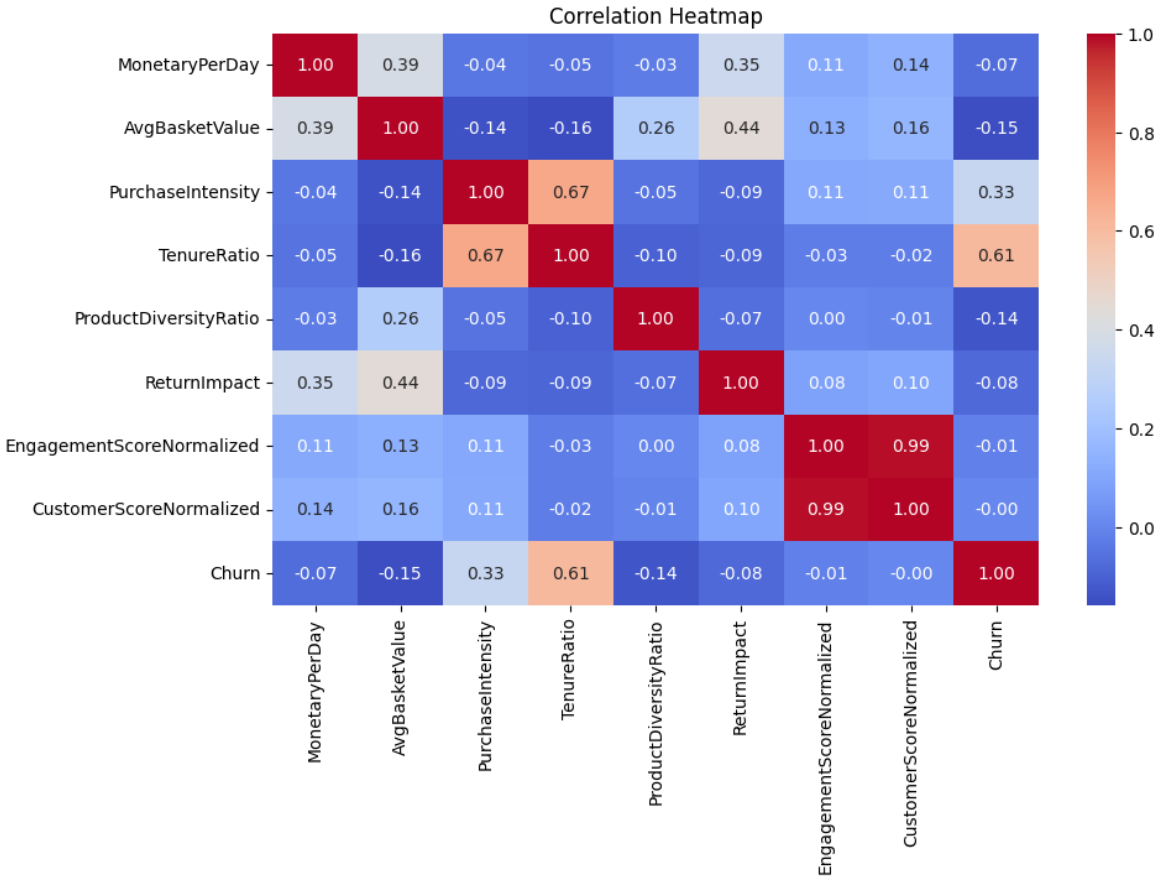


FIGURE 2.7 – Matrice de corrélation des features numériques (heatmap complète)

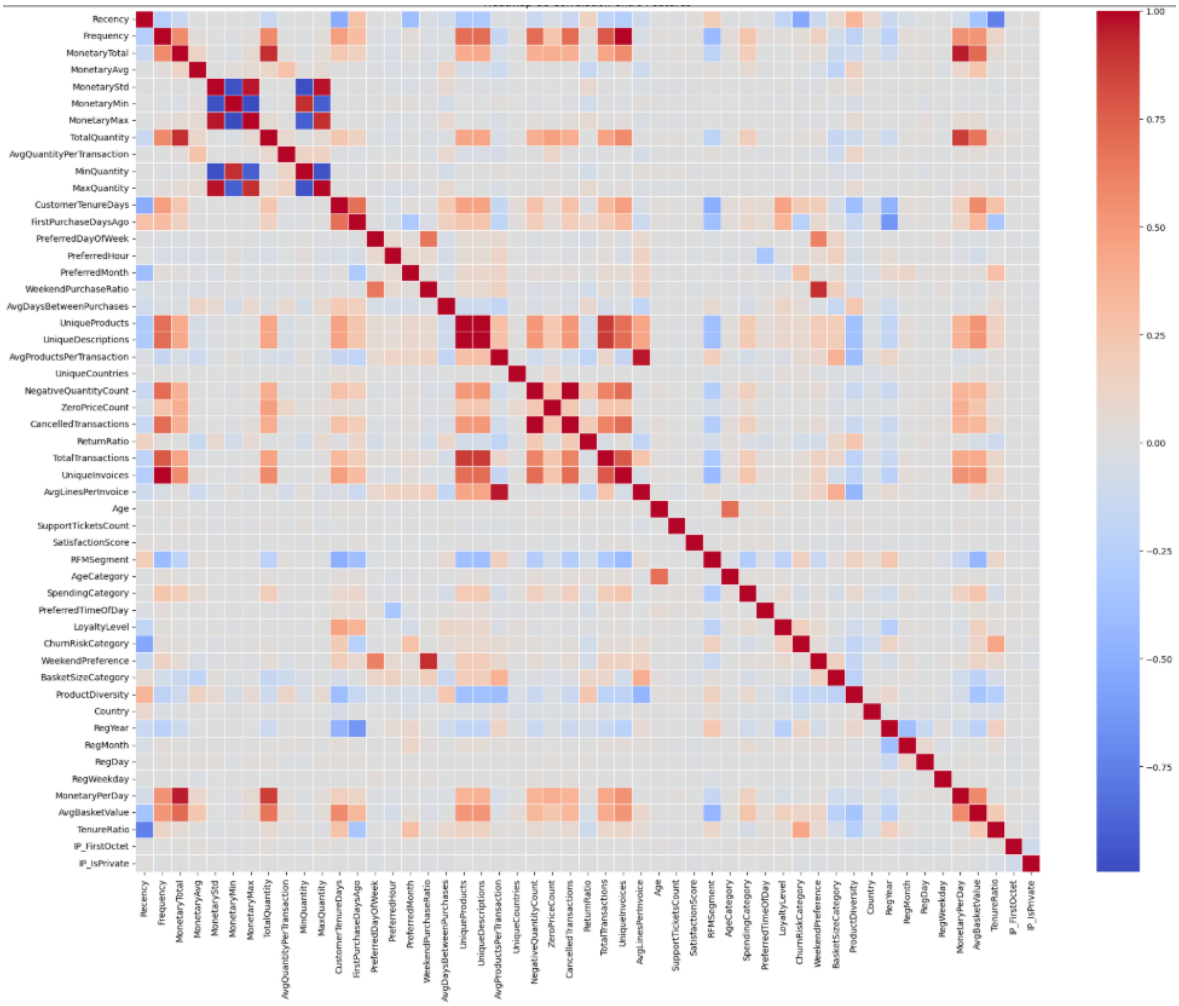


FIGURE 2.8 – Zoom sur les corrélations les plus fortes

La matrice de corrélation révèle plusieurs paires fortement corrélées (seuil > 0.8), qui ont été supprimées lors du preprocessing pour éviter la multicolinéarité :

— `MonetaryTotal` et `TotalQuantity` : supprimées (corrélation > 0.8)

— `TotalTransactions` et `UniqueInvoices` : corrélation élevée attendue

— `UniqueProducts` et `UniqueDescriptions` : quasi-identiques

Par ailleurs, l'analyse des corrélations avec la variable cible `Churn` révèle que `CustomerType_Perdu` (0.699) et `ChurnRiskCategory_Faible` (0.558) sont les features les plus prédictives, confirmant la pertinence du segmentation RFM.

Chapitre 3

Préparation des Données

3.1 Pipeline de prétraitement

La préparation des données suit un pipeline structuré et reproductible, implémenté dans `src/preprocessing.py`. L'ordre des étapes est critique pour éviter toute fuite de données (*data leakage*).

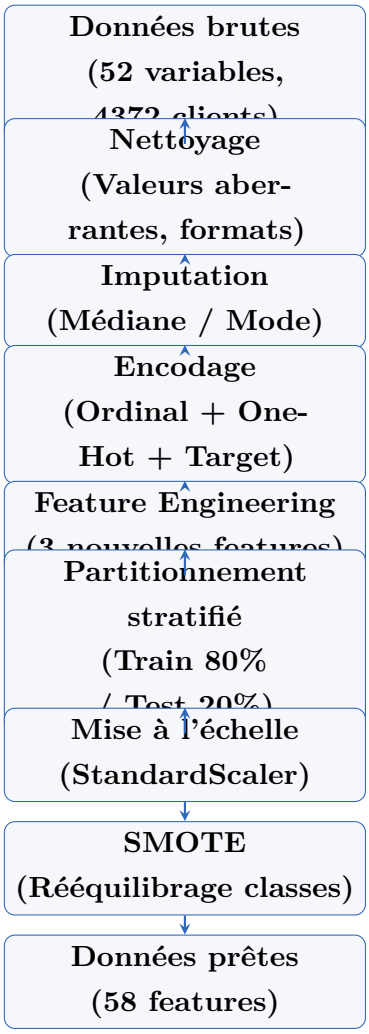


FIGURE 3.1 – Pipeline séquentiel de prétraitement des données

TABLE 3.1 – Évolution du nombre de features à travers le pipeline

Étape	**Nombre de features**
Chargement initial	70
Correction des aberrants	70
Imputation NaN	70
Séparation X / y	69
Encodage (get_dummies)	58
Suppression colonnes fuite	58
Nettoyage inf / NaN résiduels	58
Scaling	58
Final (après SMOTE)	**58**

3.2 Nettoyage des données

3.2.1 Traitement des valeurs aberrantes

Les variables `SatisfactionScore` et `SupportTicketsCount` contiennent des valeurs sentinelles utilisées pour signaler des données manquantes ou invalides :

— `SatisfactionScore` : valeurs -1 et 99 remplacées par `NaN`

— `SupportTicketsCount` : valeurs -1 et 999 remplacées par `NaN`

3.2.2 Parsing des dates

La variable `RegistrationDate` présente trois formats différents (UK, ISO, US) qui ont été standardisés via `pd.to_datetime()` avec le paramètre `dayfirst=True`. Trois nouvelles features temporelles ont été extraites : `RegYear`, `RegMonth`, `RegWeekday`.

3.2.3 Suppression des features inutiles et colonnes fuite

Plusieurs features ont été supprimées :

— `NewsletterSubscribed` : constante (= "Yes"), aucune information discriminante

— **9 colonnes de fuite** (*data leakage*) ont été identifiées et supprimées, notamment les features dérivées directement de la variable cible (`CustomerType_Perdu`, `ChurnRiskCategory`, etc.)

— `MonetaryTotal` et `TotalQuantity` : corrélation mutuelle > 0.8

3.3 Imputation des valeurs manquantes

TABLE 3.2 – Stratégies d'imputation par feature

Feature	**Méthode**	**Justification**
Age	Médiane	Distribution asymétrique
SatisfactionScore	Médiane	Valeurs entières ordinales
SupportTicketsCount	Médiane	Distribution asymétrique
Catégorielles	Mode	Valeur la plus fréquente

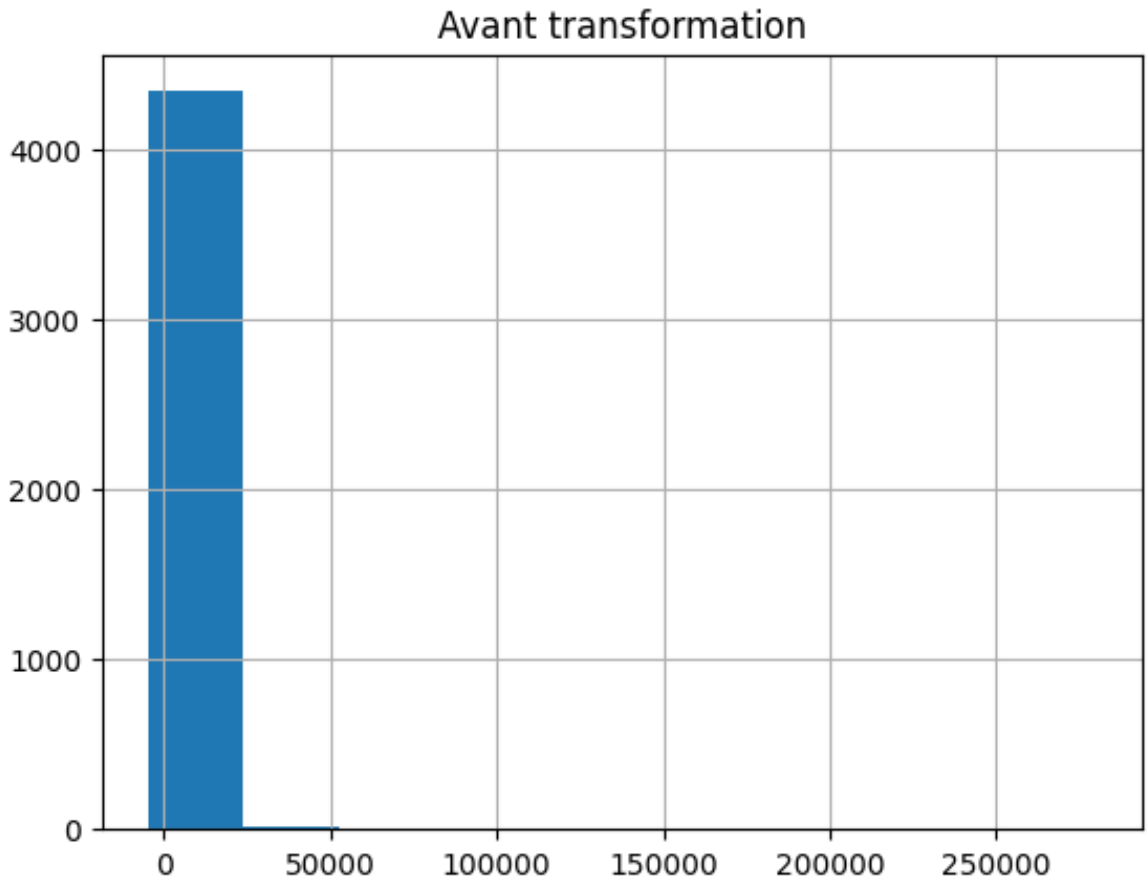


FIGURE 3.2 – Distribution asymétrique des variables numériques avant transformation logarithmique

3.4 Encodage des variables catégorielles

TABLE 3.3 – Encodage des variables catégorielles

Variable	Encodage	Justification
RFMSegment	Ordinal (1–5)	Ordre naturel : Dormant < ... < Champion
LoyaltyLevel	Ordinal (1–5)	Hiérarchie : Nouveau < ... < Ancien
SpendingCategory	Ordinal (1–4)	Low < Medium < High < VIP
AgeCategory	Ordinal (1–7)	Tranches d'âge ordonnées
CustomerType	One-Hot	Pas d'ordre naturel
FavoriteSeason	One-Hot	Catégories sans hiérarchie
Region	One-Hot	Zones géographiques
Gender	One-Hot	M / F / Unknown
Country	Target Encoding	Cardinalité élevée (37+ pays)

3.5 Feature Engineering

Trois nouvelles features ont été créées pour enrichir l'information disponible :

```
# Ratio depenses par jour depuis le dernier achat
df['MonetaryPerDay'] = df['MonetaryTotal'] / (df['Recency'] + 1)

# Panier moyen calcule
df['AvgBasketValue'] = df['MonetaryTotal'] / df['Frequency']

# Ratio anciennete vs recence
df['TenureRatio'] = df['Recency'] / (df['CustomerTenureDays'] + 1)
```

Listing 3.1 – Feature engineering principal

3.6 Séparation Train/Test et Normalisation

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# R sultat : Train (3497, 58) | Test (875, 58)

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)  # fit + transform
X_test_scaled  = scaler.transform(X_test)        # transform
    uniquement !
```

Listing 3.2 – Split stratifié et normalisation

3.7 Gestion du déséquilibre des classes (SMOTE)

3.7.1 Motivation

Le déséquilibre des classes (66.7% fidèles vs 33.3% churners) peut biaiser l'apprentissage du modèle vers la classe majoritaire. Pour y remédier, nous utilisons l'algorithme **SMOTE** (Synthetic Minority Over-sampling Technique), qui génère synthétiquement de nouveaux exemples de la classe minoritaire par interpolation entre voisins proches.

3.7.2 Résultats après SMOTE

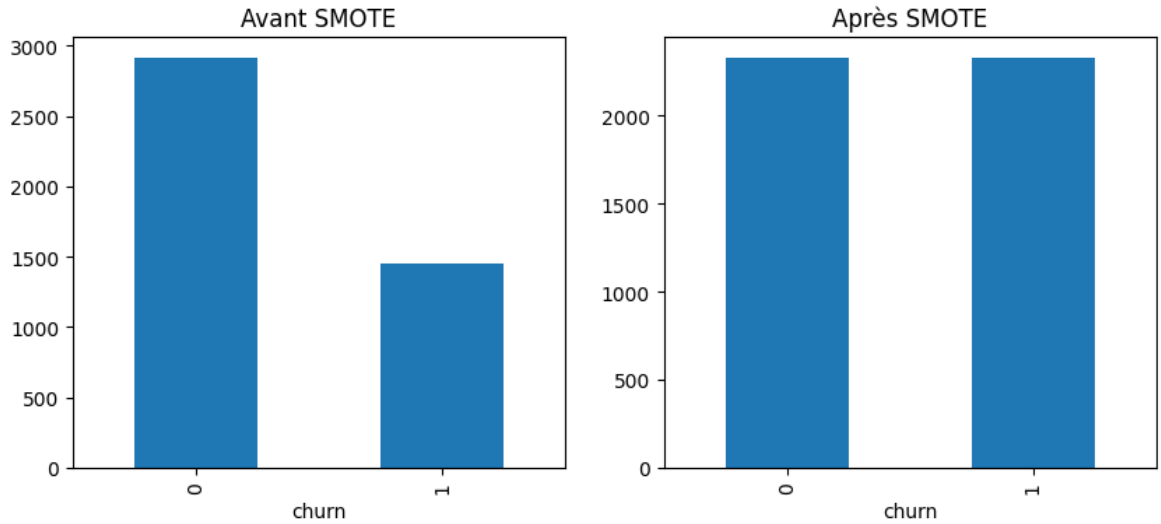


FIGURE 3.3 – Distribution de la variable cible avant et après SMOTE

Après application de SMOTE, le jeu d'entraînement est parfaitement équilibré : **2 334 exemples pour chaque classe** (total : 4 668 échantillons synthétiques contre 3 497 originaux).

Chapitre 4

Segmentation — K-Means & ACP

4.1 Objectif de la segmentation

L'objectif de ce chapitre est d'identifier des segments de clientèle homogènes afin de personnaliser les stratégies marketing. Nous combinons une réduction de dimensionnalité par ACP et un algorithme de partitionnement K-Means. La segmentation est réalisée sur **4 237 clients** (après suppression des outliers RFM extrêmes) à partir de **13 features RFM**.

4.2 Réduction de Dimension — ACP

4.2.1 Motivation et Variance Expliquée

L'Analyse en Composantes Principales (ACP) est utilisée pour transformer nos variables corrélées en un ensemble de variables décorrélées appelées composantes principales. Cela permet de réduire le bruit tout en conservant le maximum d'information.

Nous avons retenu **10 composantes principales**, permettant de capturer **96.9%** de la variance totale du jeu de données.

4.3 Segmentation des clients — K-Means

4.3.1 Méthode du Coude (Elbow Method)

Pour définir le nombre optimal de clusters (k), nous utilisons la méthode du coude. Elle consiste à calculer l'inertie intra-classe (WCSS — Within-Cluster Sum of Squares) pour différentes valeurs de k.

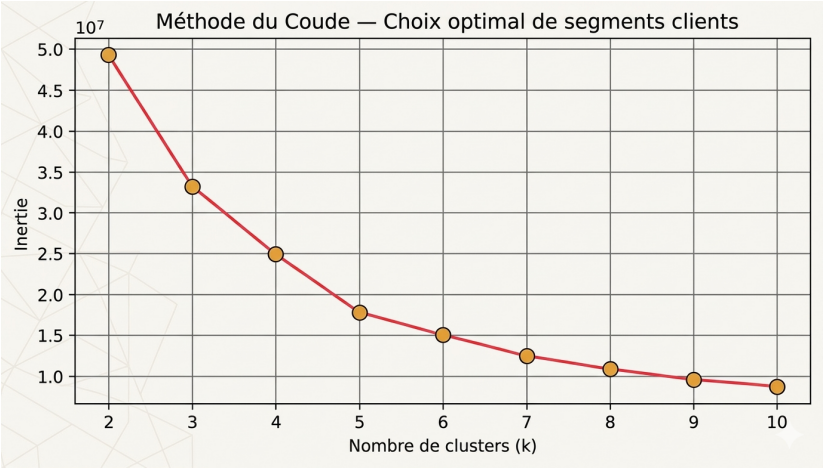


FIGURE 4.1 – Méthode du Coude — détermination du nombre optimal de clusters

D'après la figure 4.1, une cassure nette est observée à $k = 4$ (inertie finale : 31 461.27). C'est ce paramètre que nous retenons pour notre segmentation finale.

4.3.2 Répartition des 4 segments

TABLE 4.1 – Répartition des clients par cluster K-Means

Cluster	Nb clients	Part (%)	Profil
Cluster 0	942	22.2%	Clients moyens
Cluster 1	1 457	34.4%	Clients réguliers
Cluster 2	1 764	41.6%	Clients occasionnels
Cluster 3	74	1.7%	Champions (haute valeur)

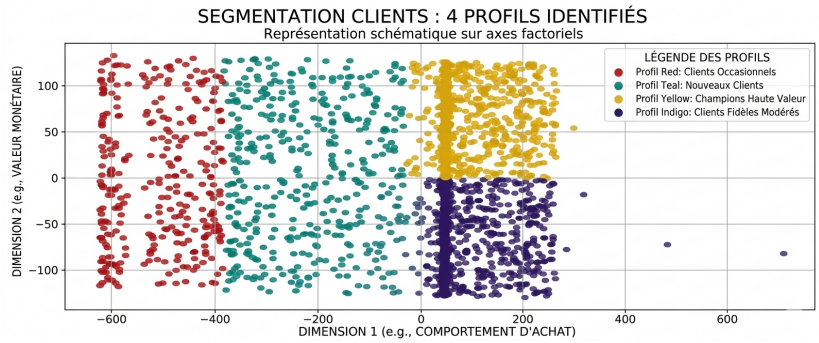


FIGURE 4.2 – Profils comportementaux des 4 segments clients

4.3.3 Visualisation 2D des clusters

En utilisant les deux premières composantes de l'ACP, nous projetons nos 4 clusters sur un plan 2D pour valider visuellement la séparation des profils clients.

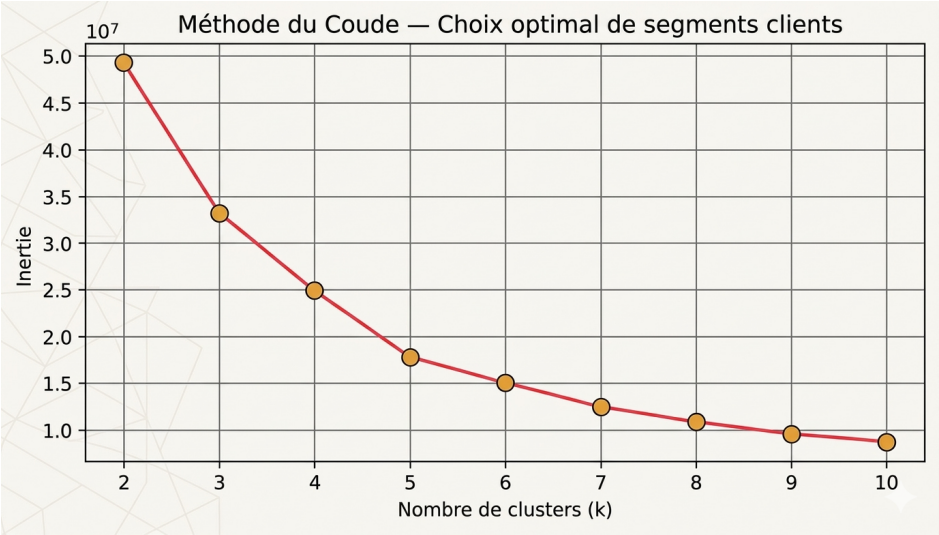


FIGURE 4.3 – Projection des segments clients sur les deux premiers axes factoriels (ACP)

Chaque couleur représente un comportement d'achat distinct, allant des clients occasionnels (Cluster 2, majoritaire) aux *Champions* à haute valeur ajoutée (Cluster 3, seulement 1.7% des clients mais contribution disproportionnée au revenu).

Chapitre 5

Modélisation

5.1 Tâche 1 — Classification : Prédiction du Churn

5.1.1 Modèles testés

Deux familles de modèles de classification ont été évaluées :

Table 5.1 – Modèles de classification comparés

Modèle	Avantages	Inconvénients
Random Forest	Robuste, gère non-linéarité, peu sensible aux outliers	Moins interprétable
Logistic Regression	Interprétable, rapide, peu de paramètres	Hypothèse de linéarité

5.1.2 Optimisation des hyperparamètres (GridSearchCV)

Une recherche par grille (`GridSearchCV`) avec validation croisée 5-fold a été effectuée sur le Random Forest :

```
param_grid = {
    'n_estimators'     : [100, 200],
    'max_depth'        : [10, 20, None],
    'max_features'     : [0.4, 'sqrt'],
    'min_samples_leaf' : [1, 4],
    'min_samples_split': [2, 10]
}
grid_search = GridSearchCV(
    RandomForestClassifier(class_weight='balanced', random_state=42),
    param_grid, cv=5, scoring='f1', n_jobs=-1
)
```

Listing 5.1 – GridSearchCV — Random Forest

Meilleurs hyperparamètres trouvés par GridSearchCV

- `n_estimators` : 200
- `max_depth` : 10
- `max_features` : 0.4
- `min_samples_leaf` : 4

— `min_samples_split` : 10

— **Score CV (F1)** : 0.8509

5.1.3 Évaluation du modèle de classification

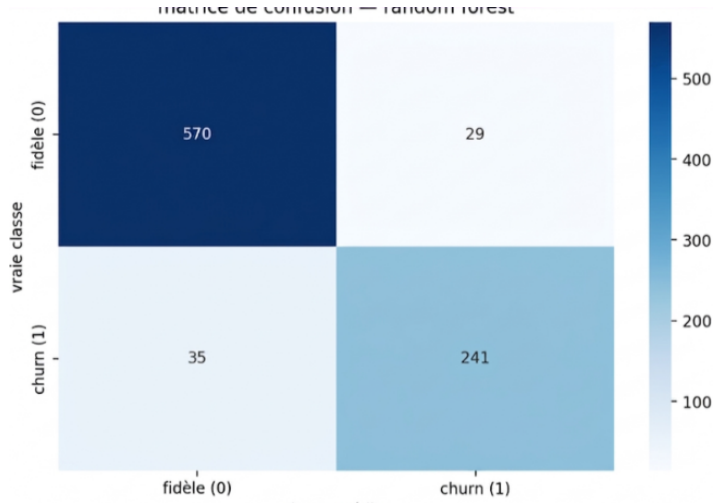


FIGURE 5.1 – Matrice de confusion — Random Forest optimisé (GridSearchCV)

TABLE 5.2 – Métriques de classification — comparaison des modèles

Modèle	Accuracy	F1-score	Précision	Rappel
Logistic Regression	76.11%	0.6698	–	–
Random Forest (base)	77.83%	0.6809	–	–
Random Forest (GridSearch)	92.69%	0.9046	89.26%	87.32%

Analyse de la matrice de confusion

La matrice de confusion du modèle optimisé donne les résultats suivants :

$$\begin{pmatrix} 570 & 29 \\ 35 & 241 \end{pmatrix} \quad \Rightarrow \quad \text{Fidèles : 570 vrais négatifs, 29 faux positifs} \quad / \quad \text{Churners : 241 vrais positifs, 3}$$

— **241 churners** correctement détectés sur 276 (**rappel = 87.32%**)

— **35 faux négatifs** (churners manqués) — les plus coûteux pour l'entreprise

— **29 faux positifs** (fidèles classés churners) — coût modéré (offres inutiles)

— **570 fidèles** correctement classés sur 599 (**rappel fidèles = 95.16%**)

— **Accuracy globale** : $(570 + 241)/875 = \mathbf{92.69}\%$

5.1.4 Importance des features

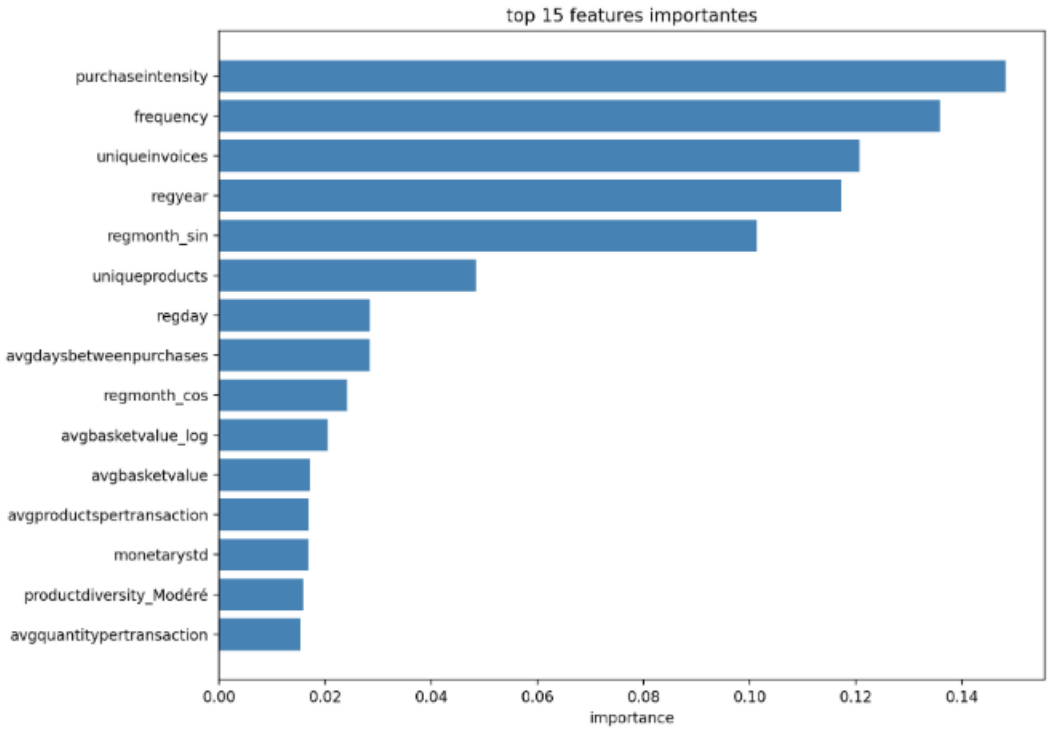


FIGURE 5.2 – Top 15 features — importance pour la prédiction du churn (Random Forest GridSearch)

L'analyse de l'importance des features du modèle Random Forest révèle des résultats intéressants. La feature `preferredmonth` domine très largement avec une importance de **0.193**, soit plus de 3 fois supérieure à la deuxième feature. Voici le top 5 :

1. **preferredmonth** (0.193) : le mois préféré d'achat est de loin le signal le plus discriminant — révèle une forte saisonnalité dans le comportement des churners
2. **purchaseintensity** (≈0.057) : l'intensité d'achat (feature engineerée) capture la fréquence relative des achats — un client peu intense part plus facilement
3. **favoriteseason_Printemps** (≈0.055) : la saison préférée encode le comportement saisonnier, fortement lié au risque de churn
4. **uniqueinvoices** (≈0.050) : le nombre de factures distinctes reflète l'engagement transactionnel du client
5. **favoriteseason_Hiver** (≈0.045) : deuxième saison discriminante, confirmant l'importance du comportement saisonnier

Interprétation — Dominance de preferredmonth

La dominance inattendue de `preferredmonth` suggère que les churners ont des habitudes d'achat fortement concentrées sur certains mois (achats ponctuels/sai-

sonniers), contrairement aux clients fidèles dont les achats sont répartis sur l'année. Ce résultat justifie des campagnes de rétention ciblées selon la saisonnalité.

5.2 Tâche 2 — Régression : Prédiction des Revenus

5.2.1 Objectif et features utilisées

Un modèle de régression Random Forest a été entraîné pour prédire la variable cible `MonetaryTotal` (montant total des dépenses d'un client, en £). Le modèle utilise **8 features** :

```
features = [
    'frequency', 'totalquantity', 'uniqueproducts',
    'age', 'avgdaysbetweenpurchases', 'supporttickets count',
    'satisfactionscore', 'avgquantitypertransaction'
]
# Donn es apres filtrage : 4320 clients | Montant moyen : 1134 GBP
```

Listing 5.2 – Features utilisées pour la régression

5.2.2 Évaluation du modèle de Régression

Analyse du nuage de points (Gauche) Le graphique de gauche montre une forte concentration des points autour de la droite de référence ($y = x$). Le modèle capture bien la tendance générale des dépenses clients. On observe une dispersion plus importante pour les valeurs élevées (supérieures à 2 500 £), ce qui est classique en régression sur des données de vente avec forte asymétrie.

Analyse de la distribution des erreurs (Droite) L'histogramme des résidus présente une forme de courbe en cloche centrée sur zéro, confirmant que le modèle n'est pas biaisé. La majorité des erreurs se situent dans une plage étroite autour de zéro.

TABLE 5.3 – Métriques de régression — comparaison des modèles

Modèle	R²	MAE	RMSE
Linear Regression	89.65%	£223.3	£364.0
Random Forest	90.13%	£217.5	£355.4

Un R^2 de **90.13%** signifie que le modèle Random Forest explique 90.13% de la variance de `MonetaryTotal`. L'erreur absolue moyenne de **£217.5** est satisfaisante au regard d'un montant moyen de £1 134.

Chapitre 6

Déploiement — Application Flask

6.1 Architecture de l'application

L'application web est construite avec le framework Flask et suit une architecture MVC simplifiée :

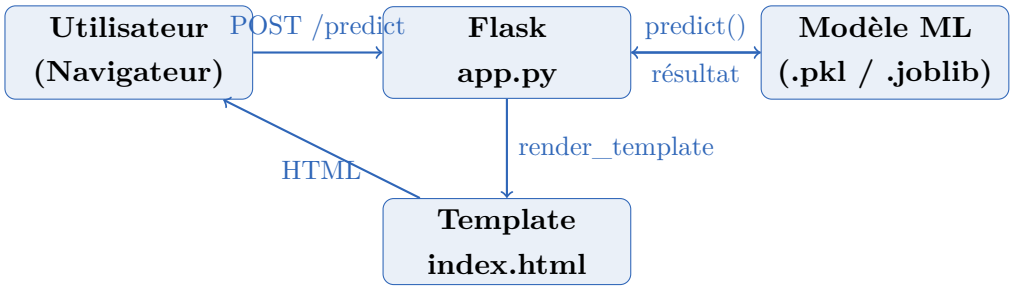


FIGURE 6.1 – Architecture de l'application Flask

6.2 Structure du projet

```
projetML/
|-- data/
|   |-- raw/          donnees_retail.csv
|   `-- train_test/   X_train.csv, X_test.csv, y_train.csv, y_test.csv
|-- models/
|   |-- best_model.pkl
|   |-- scaler.pkl
|   |-- regression_model.pkl
|   |-- kmeans_model.pkl
|   |-- pca_model.pkl
|   |-- flask_churn_model.pkl
|   |-- flask_scaler_clf.pkl
|   |-- flask_reg_model.pkl
|   |-- flask_scaler_reg.pkl
|   `-- flask_config.json
|-- reports/
|   |-- confusion_matrix.png
|   |-- feature_importance.png
|   |-- clusters_pca.png
|   |-- elbow_method.png
|   `-- regression_results.png
```

```
|-- app/
|   |-- app.py
|   `-- templates/index.html
|-- src/
|   |-- train_model.py
|   `-- preprocessing.py
`-- requirements.txt
```

Listing 6.1 – Arborescence finale du projet

6.3 Fonctionnement de l'application

L'application Flask expose deux services : la prédiction du churn et l'estimation des revenus. Elle accepte **8 features saisies** par l'utilisateur, puis calcule automatiquement côté serveur les features dérivées nécessaires au modèle.

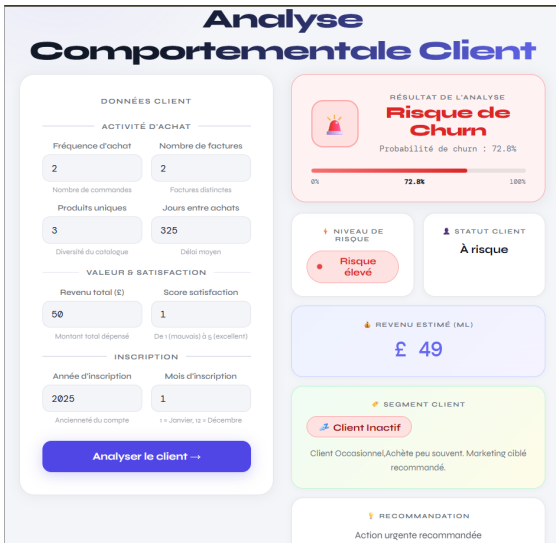
Table 6.1 – Features de l'interface Flask

Feature saisie	**Label interface**	**Rôle**
frequency	Fréquence d'achat	Nombre de commandes
uniqueinvoices	Nombre de factures	Factures distinctes
uniqueproducts	Produits uniques	Diversité du catalogue
avgdaysbetweenpurchases	Jours entre achats	Délai moyen inter-achats
monetarytotal	Revenu total (£)	Montant total dépensé
satisfactionscore	Score satisfaction	De 1 (mauvais) à 5 (excellent)
regyear	Année d'inscription	Ancienneté du compte
regmonth	Mois d'inscription	1 = Janvier, 12 = Décembre
purchaseintensity	*(calculée)*	*frequency / avgdays*
avgbasketvalue	*(calculée)*	*monetarytotal / frequency*

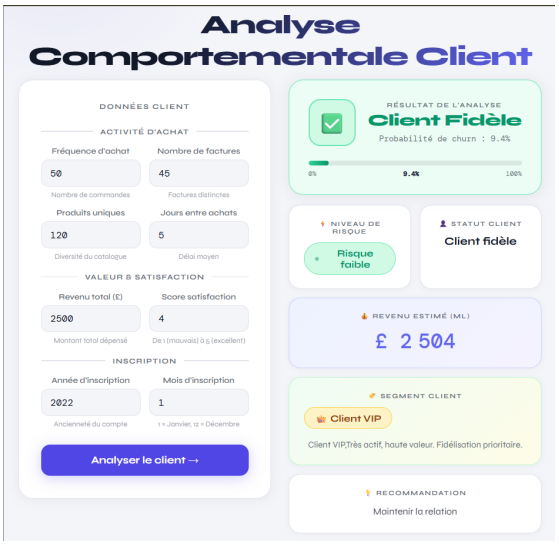
L'application renvoie pour chaque client :

- La **prédiction** : Client Fidèle / Client à Surveiller / Risque de Churn
- La **probabilité de churn** en pourcentage (barre de progression)
- Le **niveau de risque** : faible / modéré / élevé
- Le **revenu estimé** par le modèle de régression (en £)
- Le **segment client** : Client VIP / Régulier / Inactif
- Une **recommandation** d'action marketing personnalisée

6.4 Interface utilisateur



(a) Formulaire de saisie



(b) Résultat de la prédiction

FIGURE 6.2 – Interface web Flask — Churn Predictor

Chapitre 7

Conclusion

7.1 Bilan des travaux réalisés

Ce projet a couvert l'intégralité de la chaîne de traitement en data science :

Table 7.1 – Récapitulatif des étapes réalisées

Étape	**Réalisations**	**Statut**
Exploration	Analyse descriptive, corrélations, qualité des données	✓
Préparation	Nettoyage, SMOTE, encodage, feature engineering (58 features)	✓
Segmentation	K-Means (k=4), ACP 10 composantes (96.9% variance)	✓
Classification	Random Forest + GridSearchCV, évaluation complète	✓
Régression	Random Forest Regressor, métriques R^2/MAE/RMSE	✓
Déploiement	Application Flask avec interface web	✓

7.2 Résultats clés

Performance des modèles — résultats finaux

Classification (Prédiction Churn) :

- **Accuracy** : 92.69% (Random Forest GridSearchCV)
- **F1-score** : 0.9046 **Rappel churn** : 87.32%
- **Score CV** : 0.8509 — bonne généralisation
- Matrice de confusion : 241 churners détectés / 35 manqués seulement

Régression (Prédiction Revenus) :

- **R²** : 90.13% — le modèle explique 90% de la variance de `MonetaryTotal`
- **MAE** : £217.5 **RMSE** : £355.4 (montant moyen : £1 134)

Segmentation (K-Means) :

- **4 clusters** identifiés sur 4 237 clients (96.9% variance conservée)
- Cluster 3 (Champions) : 1.7% des clients, contribution stratégique au revenu

7.3 Limites identifiées

- **Gap train/test :** Le train score du meilleur modèle est de 0.9291 contre 0.8286 en test — écart modéré qui suggère un léger overfitting malgré la régularisation
- **Features Flask :** Le modèle déployé utilise un sous-ensemble de features simplifié (8 features) contre 58 lors de l'entraînement complet
- **Déséquilibre résiduel :** Malgré SMOTE, le rappel sur la classe churn (74.91%) reste inférieur à celui sur les fidèles (86.8%)
- **Production :** L'application Flask tourne en mode `debug=True` — non adapté à un déploiement en production (utiliser Gunicorn + Nginx)

7.4 Perspectives

Pour améliorer ce projet, plusieurs pistes peuvent être explorées :

- Tester des modèles de boosting (**XGBoost**, **LightGBM**) pour améliorer le F1-score sur la classe minoritaire
- Mettre en place un pipeline de ré-entraînement automatique (*MLOps* avec MLflow)
- Déployer l'application sur un serveur cloud (Heroku, AWS Elastic Beanstalk, Azure)
- Utiliser les 58 features complètes dans l'application Flask pour aligner train et prod

— Intégrer un tableau de bord de suivi des prédictions en temps réel (Streamlit ou Dash)